

INDEX

Name Aushish Neupane

Standard 4A Roll No. 1BF14C004

Subject OS

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
1	24/02/2026	To find second smallest element in array	-	
2	24/02/2026	To find sum of left diagonal of a matrix.	9	SK
3	3/03/2026	to stimulate cpu scheduling operation.	9	10/3
4	17/03/2026	Round-robin	8	17/3
5	25/03/2026	Multilevel-que.	9	24/3
6	31/03/2026	RMS and EDF	9	24/3
7	23/04/2026	Banked buffer.	SK	
8	23/04/2026	Dining philosopher	10	31/3
9	28/04/2026	Bankers(safety)	10	24/3
10		Deadlock Detection	7	SK


```

for (i=2; i<n; i++)
{
    if (arr[i] < smallest)
    {
        secSmallest = smallest;
        smallest = arr[i];
    }
    else if (arr[i] < secSmallest && arr[i]
            != smallest)
    {
        secSmallest = arr[i];
    }
}

if (smallest == secSmallest)
{
    printf("There is no second smallest
    element all elements are equal.\n");
}
else
{
    printf("The second smallest element is:
    %d\n", secSmallest);
}

return 0;
}
    
```

Output

Enter The n
Array must
*

Enter The
Enter 3

2

2

There is
elements

Enter The

Enter 4

2

1

4

The sec

Week-1

To find second smallest element in an array.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, smallest, secSmallest;
```

```
    printf("Enter the no of elements in array : ");
```

```
    scanf("%d", &n);
```

```
    if (n < 2) {
```

```
        printf("Array must have at least two elements.\n");
```

```
        return 1;
```

```
    }
```

```
    int arr[n];
```

```
    printf("Enter %d elements : \n", n);
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    if (arr[0] < arr[1])
```

```
    {
```

```
        smallest = arr[0];
```

```
        secSmallest = arr[1];
```

```
    }
```


To find The sum of The left diagonals of a matrix.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n,i,j; sum=0;
```

```
    printf("enter The size of The square matrix");
```

```
    scanf("%d",&n);
```

```
    int matrix[n][n];
```

```
    printf("Enter %d x %d matrix elements:\n",  
           n,n);
```

```
    for(i=0; i<n; i++)
```

```
    {
```

```
        for(j=0; j<n; j++)
```

```
            scanf("%d",&matrix[i][j]);
```

```
        }
```

```
    }
```

```
    for(i=0; i<n; i++)
```

```
    {
```

```
        sum += matrix[i][i];
```

```
    }
```

```
    printf("sum of left diagonal (primary  
    diagonal) elements : %d\n", sum);
```

```
    return 0;
```

```
}
```


Output

Enter the number of elements in array: 1

Array must have at least two elements.

_____ * _____ * _____ * _____

Enter The number of elements in array: 3

Enter 3 elements : 2

2

2

There is no second Smallest element all elements are equal

_____ * _____ * _____ * _____

Enter The number of elements in array: 4

Enter 4 elements: 3

2

1

4

The second smallest element is 2.

Output

Enter the size of the square matrix: 2

Enter 2x2 matrix elements:

3

5

2

7

Sum of left diagonal (primary diagonal) element: 10

sk
wh

Week 2

To simulate CPU scheduling algorithm

1 FCFS

2 SJF

code

#include <stdio.h>

#define MAX 20

void fcs(int n, int at[], int bt[]) {

int wt[MAX], t at[MAX],

int time = 0;

float total wt = 0, total tat = 0;

for (int i = 0; i < n; i++)

{ if (time < at[i])

{

time = at[i];

rt[i] = time - at[i];

wt[i] = time - at[i]; wt[i] = rt[i];

time += bt[i]; at[i] = time;

tat[i] = at[i] - at[i];

total wt += wt[i];

total tat += tat[i];

total - rt += rt[i];

}

printf ("n --- FCS --- n");

printf ("P1 D) TAT + BT + C - AT) \n

+ RT \n");

for (int i = 0; i < n; i++)

{ printf ("P1 D) TAT + BT + C - AT) \n

+ RT \n");

}

printf ("n --- FCS --- n");

printf ("P1 D) TAT + BT + C - AT) \n

+ RT \n");

}

printf ("n --- FCS --- n");

printf ("P1 D) TAT + BT + C - AT) \n

+ RT \n");

}

printf ("n --- FCS --- n");

printf ("P1 D) TAT + BT + C - AT) \n

+ RT \n");

}

void

{

int at

rt[m

int a

int r

float

while

int

for (i

if

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

```
for(cnt = 0; i < n; i++)  
    {  
        printf("%d\t %d\t %d\t %d\t %d\t %d\t\n", at[i], bt[i], rt[i], tot[i], wt[i], ttt[i]);  
        total_wt += n;  
        printf("\n Average Waiting Time = %d", total_wt / n);  
        total_tat += n;  
        printf("\n Average Turnaround Time = %d", total_tat / n);  
        total_rt += n;  
        printf("\n Average Response Time = %d", total_rt / n);  
    }
```

```

int wt[MAY] = {0}, tat[MAY] = {0}, ct[MAY]
rt[MAY] = {0};
int completed[MAY] = {0};
int time = 0, count = 0;
float total_wt = 0, total_tat = 0, total_rt = 0;
while (count < n) {
    int min = 999, idx = -1;
    for (int i = 0; i < n; i++) {
        if (ct[i] <= time && !completed[i]) {
            if (bt[i] < min) {
                min = bt[i];
                idx = i;
            }
        }
    }
}

```

3

3


```

int main() {
    int n, choice, at [MAX], bt [MAX];
    printf ("Enter number of processes\n");
    scanf ("%d", &n);
    for (int i = 0; i < n; i++) {
        printf ("\n process: %d\n", i+1);
        printf ("Arrival Time: ");
        scanf ("%d", &at[i]);
        printf ("Burst Time: ");
        scanf ("%d", &bt[i]);
    }

    printf ("\n Choose Algorithm: ");
    printf ("\n 1. FCFS\n 2. SJF-Non-preemptive\n 3. SJF-Preemptive (SRTF)");
    printf ("\n Enter choice: ");
    scanf ("%d", &choice);

```

```

    switch (choice) {
        case 1:
            fcs(n, at, bt);
            break;
        case 2:
            sjf-nonpreemptive(n, at, bt);
            break;
        case 3:
            sjf-preemptive(n, at, bt);
            break;
        default:
            printf ("Invalid choice");
    }

    return 0;
}

```


Enter choice: 1

--- FIFS ---

PID	AT	BT	CT	TAT	WT	RT
1	0	5	5	5	0	0
2	4	7	12	8	1	1
3	3	3	15	12	9	9
4	6	9	24	18	9	9

Average Waiting Time: 4.75

Average Turnaround Time: 10.75

Average Response Time: 4.75

Enter choice: 2

--- SJF Non-Preemptive ---

PID	AT	BT	CT	TAT	WT	RT
1	0	5	5	5	0	0
2	4	7	15	11	4	4
3	3	3	8	5	2	2
4	6	9	24	18	9	9

Average Waiting Time: 3.75

Average Turnaround Time: 9.75

Average Response Time: 3.75

Enter choice: 3

--- SJF preemptive ---

PID	AT	BT	CT	TAT	WT	RT
1	0	5	5	5	0	0
2	4	7	15	11	4	4
3	3	3	8	5	2	2
4	6	9	24	18	9	9

Average waiting time = 3.75

Average Turnaround Time = 9.75

Average Response Time = 3.75

Comparison:

SJF preemptive is the best, then SJF Non-preemptive, and then FCFS is slowest.

✓
16/3

Output

Enter number of processes : 4

process 1

Arrival Time : 0

Burst Time : 5

process 2

Arrival Time : 4

Burst Time : 7

process 3

Arrival Time : 3

Burst Time : 3

process 4

Arrival Time : 6

Burst Time : 9

Choose Algorithm:

1. FCFS

2. SJF Non-preemptive

3. SJF Preemptive (SRIF)

Enter choice

- - - FCFS

PID AT

1

2

3

4

Average

Average

Average

Enter choice

- - - SJF

PID AT

1

2

3

4

Average

Average

Average

```

ct[idn] = time;
tat[idn] = ct[idn] - at[idn];
wt[idn] = tat[idn] - bt[idn];

int response = start[idn] - at[idn];

sumTAT = tat[idn];
sumWT = wt[idn];
sumRT = response;

completed++;
}

printf("\n\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i < n; i++) {
    printf("\nP%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
           i/d, i%10, bt[i], pr[i], ct[i], tat[i], wt[i],
           start[i] - at[i]);
}

printf("\n\nAverage TAT : %.2f", sumTAT/n);
printf("\n\nAverage WT : %.2f", sumWT/n);
printf("\n\nAverage RT : %.2f\n", sumRT/n);

void priorityPreemptive (int id[], int at[],
                        int bt[], int pr[], int n)
{
    int rt[n], ct[n], tat[n], wt[n], start[n];
    int time = 0, completed = 0;
    float sumTAT = 0, sumWT = 0, sumRT = 0;
}

```



```

for (int i=0; i<n; i++)
{
    rt[i] = bt[i];
    ct[i] = tat[i] - wt[i] - 1;
    start[i] = -1;
}

printf("\n-- Preemptive Priority Scheduling --");
printf("\n (start 'n')");
while (completed < n)
{
    int idm = -1, highest = 1;
    for (int i=0; i<n; i++)
    {
        if (ct[i] <= time + 1 & i >= 0 + 1 & pr[i] <= highest)
        {
            highest = pr[i];
            idm = i;
        }
    }
    if (idm == -1) { time++; } else
    {
        if (start[idm] == -1)
        {
            start[idm] = time;
            printf("p: d", id[idm]);
            rt[idm]--;
            time++;
            if (rt[idm] == 0)
            {
                ct[idm] = time;
                tat[idm] = ct[idm] - at[idm];
                wt[idm] = tat[idm] - bt[idm];
                int response = start[idm] - at[idm];
                sumTAT += tat[idm];
                sumWT += wt[idm];
                sumRT += response;
                completed++;
            }
        }
    }
}

```

Week 3

Priority Scheduling

```
#include <stdio.h>
```

```
void PriorityNonPreemptive(int id[], int at[],
int bt[], int pr[], int n) {
    int ct[n], tat[n], wt[n], start[n];
    int time = 0, completed = 0;
    float sumAT = 0, sumWT = 0, sumRT = 0;
```

```
    for (int i = 0; i < n; i++)
        ct[i] = tat[i] = wt[i] = start[i] = -1;
```

```
    printf("\n-Non-Preemptive Priority
    Scheduling--");
    printf("\nGrant Chart:\n");
```

```
    while (completed < n)
```

```
    {
        int idx = -1, highest = 1e9;
```

```
        for (int i = 0; i < n; i++)
```

```
        {
            if (at[i] <= time && ct[i] == -1) {
```

```
                pr[i] < highest) {
```

```
                    highest = pr[i];
```

```
                    idx = i;
```

```
                }
```

```
            }
            if (idx == -1) { time++; } else
```

```
            { start[idx] = time;
```

```
              printf("%d\t", id[idx]);
```

```
              time += bt[idx];
```

```
              completed++;
```


ATATATAT

ATATATAT

ATATAT

ATATAT

ATATAT

ATATAT

ATATAT

ATATAT

ATATAT

ATATAT

```
printf("Enter your choice:");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        priority_NonPreemptive(id, at, bt, pr, n);
        break;
    case 2:
        priority_PriorityPreemptive(id, at, bt, pr, n);
        break;
    default:
        printf("Invalid choice!");
}

return 0;
```

Output:-

Enter number of processes: 4

Arrival Time for P1: 0

Burst Time for P1: 7

Priority for P1 (lower number = higher priority): 5

Arrival Time for P2: 2

Burst Time for P2: 2

Priority for P2 (lower number = higher priority): 3

Arrival time for P3: 3
 Burst time for P3: 5
 Priority for P3 (lower number = higher priority): 1

Arrival time for P4: 4
 Burst time for P4: 4
 Priority for P4 (lower number = higher priority): 1

CPU Scheduling Menu

1. Priority Scheduling (Non-Preemptive)
 2. Priority Scheduling (Preemptive)
- Enter your choice: 1

-- Non-Preemptive Priority Scheduling --

Gantt Chart:

	P1	P3	P2	P4
0	7	12	14	18

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	7	5	7	7	0	0
P2	2	2	3	14	12	10	10
P3	3	5	2	12	9	4	4
P4	4	4	4	18	14	10	10

Average TAT: 10.50
 Average WT: 6.00
 Average RT: 6.00

Enter your choice: 1
-- Preemptive
Gantt Chart:
P1 P3 P2 P4
0 7 12 14 18
Process
P1
P2
P3
P4
Average
Average
Average


```
printf("\n\nProcess STAT: B1 B2 B3 B4 B5 B6 B7 B8 B9 B10 B11 B12 B13 B14 B15 B16 B17 B18 B19 B20 B21 B22 B23 B24 B25 B26 B27 B28 B29 B30 B31 B32 B33 B34 B35 B36 B37 B38 B39 B40 B41 B42 B43 B44 B45 B46 B47 B48 B49 B50 B51 B52 B53 B54 B55 B56 B57 B58 B59 B60 B61 B62 B63 B64 B65 B66 B67 B68 B69 B70 B71 B72 B73 B74 B75 B76 B77 B78 B79 B80 B81 B82 B83 B84 B85 B86 B87 B88 B89 B90 B91 B92 B93 B94 B95 B96 B97 B98 B99 B100 B101 B102 B103 B104 B105 B106 B107 B108 B109 B110 B111 B112 B113 B114 B115 B116 B117 B118 B119 B120 B121 B122 B123 B124 B125 B126 B127 B128 B129 B130 B131 B132 B133 B134 B135 B136 B137 B138 B139 B140 B141 B142 B143 B144 B145 B146 B147 B148 B149 B150 B151 B152 B153 B154 B155 B156 B157 B158 B159 B160 B161 B162 B163 B164 B165 B166 B167 B168 B169 B170 B171 B172 B173 B174 B175 B176 B177 B178 B179 B180 B181 B182 B183 B184 B185 B186 B187 B188 B189 B190 B191 B192 B193 B194 B195 B196 B197 B198 B199 B200 B201 B202 B203 B204 B205 B206 B207 B208 B209 B210 B211 B212 B213 B214 B215 B216 B217 B218 B219 B220 B221 B222 B223 B224 B225 B226 B227 B228 B229 B230 B231 B232 B233 B234 B235 B236 B237 B238 B239 B240 B241 B242 B243 B244 B245 B246 B247 B248 B249 B250 B251 B252 B253 B254 B255 B256 B257 B258 B259 B260 B261 B262 B263 B264 B265 B266 B267 B268 B269 B270 B271 B272 B273 B274 B275 B276 B277 B278 B279 B280 B281 B282 B283 B284 B285 B286 B287 B288 B289 B290 B291 B292 B293 B294 B295 B296 B297 B298 B299 B300 B301 B302 B303 B304 B305 B306 B307 B308 B309 B310 B311 B312 B313 B314 B315 B316 B317 B318 B319 B320 B321 B322 B323 B324 B325 B326 B327 B328 B329 B330 B331 B332 B333 B334 B335 B336 B337 B338 B339 B340 B341 B342 B343 B344 B345 B346 B347 B348 B349 B350 B351 B352 B353 B354 B355 B356 B357 B358 B359 B360 B361 B362 B363 B364 B365 B366 B367 B368 B369 B370 B371 B372 B373 B374 B375 B376 B377 B378 B379 B380 B381 B382 B383 B384 B385 B386 B387 B388 B389 B390 B391 B392 B393 B394 B395 B396 B397 B398 B399 B400 B401 B402 B403 B404 B405 B406 B407 B408 B409 B410 B411 B412 B413 B414 B415 B416 B417 B418 B419 B420 B421 B422 B423 B424 B425 B426 B427 B428 B429 B430 B431 B432 B433 B434 B435 B436 B437 B438 B439 B440 B441 B442 B443 B444 B445 B446 B447 B448 B449 B450 B451 B452 B453 B454 B455 B456 B457 B458 B459 B460 B461 B462 B463 B464 B465 B466 B467 B468 B469 B470 B471 B472 B473 B474 B475 B476 B477 B478 B479 B480 B481 B482 B483 B484 B485 B486 B487 B488 B489 B490 B491 B492 B493 B494 B495 B496 B497 B498 B499 B500 B501 B502 B503 B504 B505 B506 B507 B508 B509 B510 B511 B512 B513 B514 B515 B516 B517 B518 B519 B520 B521 B522 B523 B524 B525 B526 B527 B528 B529 B530 B531 B532 B533 B534 B535 B536 B537 B538 B539 B540 B541 B542 B543 B544 B545 B546 B547 B548 B549 B550 B551 B552 B553 B554 B555 B556 B557 B558 B559 B560 B561 B562 B563 B564 B565 B566 B567 B568 B569 B570 B571 B572 B573 B574 B575 B576 B577 B578 B579 B580 B581 B582 B583 B584 B585 B586 B587 B588 B589 B590 B591 B592 B593 B594 B595 B596 B597 B598 B599 B600 B601 B602 B603 B604 B605 B606 B607 B608 B609 B610 B611 B612 B613 B614 B615 B616 B617 B618 B619 B620 B621 B622 B623 B624 B625 B626 B627 B628 B629 B630 B631 B632 B633 B634 B635 B636 B637 B638 B639 B640 B641 B642 B643 B644 B645 B646 B647 B648 B649 B650 B651 B652 B653 B654 B655 B656 B657 B658 B659 B660 B661 B662 B663 B664 B665 B666 B667 B668 B669 B670 B671 B672 B673 B674 B675 B676 B677 B678 B679 B680 B681 B682 B683 B684 B685 B686 B687 B688 B689 B690 B691 B692 B693 B694 B695 B696 B697 B698 B699 B700 B701 B702 B703 B704 B705 B706 B707 B708 B709 B710 B711 B712 B713 B714 B715 B716 B717 B718 B719 B720 B721 B722 B723 B724 B725 B726 B727 B728 B729 B730 B731 B732 B733 B734 B735 B736 B737 B738 B739 B740 B741 B742 B743 B744 B745 B746 B747 B748 B749 B750 B751 B752 B753 B754 B755 B756 B757 B758 B759 B760 B761 B762 B763 B764 B765 B766 B767 B768 B769 B770 B771 B772 B773 B774 B775 B776 B777 B778 B779 B780 B781 B782 B783 B784 B785 B786 B787 B788 B789 B790 B791 B792 B793 B794 B795 B796 B797 B798 B799 B800 B801 B802 B803 B804 B805 B806 B807 B808 B809 B810 B811 B812 B813 B814 B815 B816 B817 B818 B819 B820 B821 B822 B823 B824 B825 B826 B827 B828 B829 B830 B831 B832 B833 B834 B835 B836 B837 B838 B839 B840 B841 B842 B843 B844 B845 B846 B847 B848 B849 B850 B851 B852 B853 B854 B855 B856 B857 B858 B859 B860 B861 B862 B863 B864 B865 B866 B867 B868 B869 B870 B871 B872 B873 B874 B875 B876 B877 B878 B879 B880 B881 B882 B883 B884 B885 B886 B887 B888 B889 B890 B891 B892 B893 B894 B895 B896 B897 B898 B899 B900 B901 B902 B903 B904 B905 B906 B907 B908 B909 B910 B911 B912 B913 B914 B915 B916 B917 B918 B919 B920 B921 B922 B923 B924 B925 B926 B927 B928 B929 B930 B931 B932 B933 B934 B935 B936 B937 B938 B939 B940 B941 B942 B943 B944 B945 B946 B947 B948 B949 B950 B951 B952 B953 B954 B955 B956 B957 B958 B959 B960 B961 B962 B963 B964 B965 B966 B967 B968 B969 B970 B971 B972 B973 B974 B975 B976 B977 B978 B979 B980 B981 B982 B983 B984 B985 B986 B987 B988 B989 B990 B991 B992 B993 B994 B995 B996 B997 B998 B999 1000");
```

A photograph of a blank, lined notebook page. The page is white with horizontal blue lines and a vertical red margin line on the left. It is resting on a wooden desk. A hand is visible on the right side, holding the edge of the page. The bottom half of the image is a solid black rectangle.

Week 4

Round Robin

```
#include <stdio.h>

void RoundRobin (int id[], int at[], int bt[], int n, int tq) {
    int i, j, ct[n], tot[n], out[n], start[n];
    int completed = 0, time = 0;
    float sumTAT = 0, sumWT = 0, sumRT = 0;
    int gantt[200], gant[200], g = 0;
    for (int i = 0; i < n; i++) {
        ct[i] = bt[i];
        at[i] = tot[i] = out[i] = -1;
        start[i] = -1;
    }
    printf("\n--- Round Robin Scheduling ---\n");
    while (completed < n) {
        int executed = 0;
        for (int i = 0; i < n; i++) {
            if (at[i] <= time && ct[i] > 0) {
                gantt[g] = id[i];
                gant[g] = time;
                g++;
                if (ct[i] > tq) {
                    time += tq;
                    ct[i] -= tq;
                }
                else {
                    time = time + ct[i];
                    ct[i] = 0;
                    completed++;
                }
            }
        }
    }
    printf("Gantt Chart: ");
    for (int i = 0; i < g; i++) {
        printf("%d ", gantt[i]);
    }
    printf("\n");
    printf("Turnaround Time (TAT): ");
    for (int i = 0; i < n; i++) {
        printf("%d ", start[i]);
    }
    printf("\n");
    printf("Waiting Time (WT): ");
    for (int i = 0; i < n; i++) {
        printf("%d ", at[i]);
    }
    printf("\n");
    printf("Response Time (RT): ");
    for (int i = 0; i < n; i++) {
        printf("%d ", out[i]);
    }
    printf("\n");
    printf("Total TAT: ");
    printf("%f", sumTAT);
    printf("\n");
    printf("Total WT: ");
    printf("%f", sumWT);
    printf("\n");
    printf("Total RT: ");
    printf("%f", sumRT);
    printf("\n");
}
```



```

3  time += t[i],
4  t[i] = 0;
5  at[i] = time;
6  tat[i] = at[i] - at[i];
7  wt[i] = tat[i] - bt[i];
8  int response = start[i] - at[i];
9  sumTAT += tat[i];
10 sumWT += wt[i];
11 sumRT += response;
12 completed++;
13
14 executed = 1;
15
16 if (!executed)
17     time++;
18
19 gantt[i] = time;
20 printf("\n Gantt chart \n");
21 for (int i = 0; i < n; i++)
22     printf("%d ", gantt[i]);
23 printf("\n Process TAT: %d, BT: %d, RT: %d, WT: %d",
24         TAT, BT, RT, WT);
25 for (int i = 0; i < n; i++)
26     printf("\n P: %d, T: %d, D: %d, T: %d, D: %d",
27         i, t[i], d[i], t[i], d[i]);
28 id[i], at[i], bt[i], tat[i], wt[i];
29 start[i] = at[i];
30
31 printf("\n\n Average TAT: %.2f, sumTAT: %d",
32         WT, WT, sumWT);
33 printf("\n\n Average RT: %.2f, sumRT: %d",
34         sumRT, sumRT);

```

Every your choice: 7

--- Preemptive Priority Scheduling ---

Gantt Chart

| P1 | P1 | P2 | P3 | P3 | P3 | P3 | P3 | P2 | P4 | P4
 0 1 2 3 4 5 6 7 8 9 10
 P4 | P4 | P1 | P1 | P1 | P1 | P1 | P1 | P1 | P1 | P1
 11 12 13 14 15 16 17 18

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	7	5	18	18	11	0
P2	2	2	3	9	7	5	0
P3	3	5	2	8	5	0	0
P4	4	4	4	13	9	5	5

Average TAT: 9.75

Average WT: 5.25

Average RT: 1.25

8/22
1413

Arrival Time for P4 :- 1

Best Time for P4 :- 9

Arrival Time for P5 :- 2

Best Time for P5 :- 7

Arrival Time for P6 :- 6

Best Time for P6 :- 3

Enter the Quantum: 4

Gantt chart:

P4	P5	P3	P2	P1	P4	P6	P3	P2
32	33	1	2	7	23	26	29	31

Process	AT	BT	CT	TAT	WT	RT
P4	1	9	33	32	21	0
P5	2	2	7	5	3	3
P3	3	7	29	26	19	4
P2	4	6	31	27	21	7
P1	5	5	32	27	22	10
P6	6	3	16	20	17	17

Average TAT: 27.83

Average WT: 17.50

Average RT: 6.83

multilevel queuing

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

4-Johne MAX 100

voedde structe

```

int process_id;
int arrival_time;
int burst_time;
int remaining_time;
char type[10];
int completion_time;
int waiting_time;
int turnaround_time;
} process;

```

typedef struct

```
processQueue[MAX];
int front, rear;
Queue;
```

```
void initQueue(Queue *q) {
    q->front = q->rear = -1;
```

```
int isEmpty (Queue *q) {
    return q->front == -1;
}
```

void end
if (q =
if (q
q =

```

Process do
  process
  if (q = 0)
    y = 0
  else
    y = 1
  end if
end process

```

int main
int
par
see


```
void enqueue (Queue *q, process p) {
    if (q->rear == MAX-1) return;
    if (q->front == -1) q->front = 0;
    q->queue [q->rear] = p;
}
```

2)

```
process dequeue (Queue *q) {
    process p = q->queue [q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    } else {
        q->front++;
    }
    return p;
}
```

3

```
int main() {
    int n;
    printf("Enter number of processes:");
    scanf ("%d", &n);

    process process_list[MAX];
    for (int i = 0; i < n; i++) {
        printf ("Enter arrival time:");
        printf ("%d", process_list[i].arrival_time);
        scanf ("%d", &process_list[i].arrival_time);

        printf ("Enter burst time:");
        printf ("%d", &process_list[i].burst_time);
        scanf ("%d", &process_list[i].burst_time);

        printf ("Enter type (0=System, 1=User):");
        int t;
        scanf ("%d", &t);
        if (t == 0) strcpy (process_list[i].type,
            "System");
    }
}
```

```
int main()
{
    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int id[n], at[n], bt[n];
    for (int i = 0; i < n; i++)
    {
        id[i] = i + 1;
        printf("Enter Arrival Time for P%d: ", i + 1);
        scanf("%d", &at[i]);
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
}
```

```
RoundRobin (id, at, bt, n, tq);
return 0;
```

Output

Enter number of processes: 6

Arrival Time for P1: - 5

Burst Time for P1: - 5

Arrival Time for P2: - 4

Burst Time for P2: - 6

Arrival Time for P3: - 3

Burst Time for P3: - 7

Arrival
Burst

Arrival
Burst

Arrival
Burst

Enter

Grant

P4

P1

32

Process

P4

P5

P3

P2

P1

P6

Arrival

Arrival

Arrival

User")

ess list[i]

while (completed < n) {

while (i < n && process_list[i].arrival_time <= current_time) {

if (strcmp(process_list[i].type, "System") == 0) {

enqueue(&system_queue, process_list[i]);

} else {

enqueue(&user_queue, process_list[i]);

i++;

if (current_process != NULL) {

if (strcmp(current_process->type, "User") == 0 && !is_empty(&system_queue)) {

enqueue(&user_queue, *current_process);

current_process = NULL;

if (current_process == NULL) {

if (!is_empty(&system_queue)) {

Process p = dequeue(&system_queue);

current_process = (Process *) malloc(sizeof(Process));

*current_process = p;

} else {

current_time++;

continue;

```
current_process -> remaining_time = i;
gantt[gantt_index] = current_process;
    -> process_id;
gantt_time[gantt_index] = current_time;
gantt_index++;
current_time++;
```

```
if (current_process -> remaining_time == 0)
    & current_process -> completion_time =
    current_time;
```

```
for (int k=0; k<n; k++) {
    if (process_list[k].process_id == current
        - process -> completion_time;
    process_list[k].turnaround_time =
    process_list[k].completion_time -
    process_list[k].arrival_time;
    process_list[k].waiting_time =
    process_list[k].turnaround_time -
    process_list[k].burst_time;
}
```

```
3
    completed++;
    free (current_process);
    current_process = NULL;
2
3
```



```
else strcpy(process_list[i], type, "User");
```

```
process_list[i].process_id = i;
process_list[i].remaining_time = process_list[i].burst_time;
process_list[i].completion_time = 0;
process_list[i].waiting_time = 0;
process_list[i].turnaround_time = 0;
```

```
for (int i = 0; i < n; i++) {
    for (int j = 1; j < n; j++) {
        if (process_list[i].arrival_time < process_list[j].arrival_time) {
            process_list[i].temp = process_list[j].temp;
            process_list[j].temp = process_list[i].temp;
            process_list[i].temp = process_list[j].temp;
        }
    }
}
```

```

}
}
}

```

```

Queue systemQueue, userQueue;
init Queue (&systemQueue);
init Queue (&userQueue);

```

```

int current_time = 0, completed = 0, i = 0;
Process *current_process = NULL;

```

```

int gantt[MAX];
int gantt_time[MAX];
int gantt_index = 0;

```

```

while (i < n) {
    while (i < n) {
        if (process_list[i].remaining_time == 0) {
            process_list[i].remaining_time = process_list[i].burst_time;
            process_list[i].waiting_time = 0;
            process_list[i].turnaround_time = 0;
        }
        if (process_list[i].arrival_time < process_list[j].arrival_time) {
            process_list[i].temp = process_list[j].temp;
            process_list[j].temp = process_list[i].temp;
            process_list[i].temp = process_list[j].temp;
        }
    }
}

```

Output

Enter number of processes: 4

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0=System, 1=User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0=System, 1=User): 1

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0=System, 1=User): 0

Process 3

Enter arrival time: 3

Enter burst time: 2

Enter type (0=System, 1=User): 1

ID	PC	PP
0	0	0
2	1	1
1	0	0
3	1	1

Average

Average

Count

1 PC

0 PP

1 PP

8/2

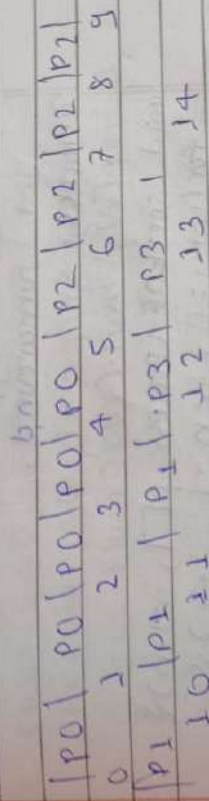
9

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
2	System	1	4	9	4	8
1	User	2	3	12	7	10
3	User	3	2	14	9	11

Average Waiting Time = 5.00

Average Turnaround Time = 8.50

Gantt Chart:



20/10/20

```

printf("\n IO Type AT BT CT WT TAT\n");
double totalWT = 0, totalTAT = 0;
for (int i = 0; i < n; i++) {
    printf("%d\t", process_list[i].arrival_time);
    printf("%d\t", process_list[i].burst_time);
    printf("%d\t", process_list[i].completion_time);
    printf("%d\t", process_list[i].waiting_time);
    printf("%d\t", process_list[i].turnaround_time);
    totalWT += process_list[i].waiting_time;
    totalTAT += process_list[i].turnaround_time;
}

printf("\n Average Waiting Time: %.2f\n",
totalWT / n);
printf("\n Average Turnaround Time: %.2f\n", totalTAT / n);

printf("\n Gantt Chart:\n");
for (int j = 0; j < gantt_index; j++) {
    printf("%d\t", process_list[j]);
}

printf("\n");
for (int j = 0; j < gantt_index; j++) {
    printf("%d\t", gantt_time[j]);
}

printf("%d\t", current_time);
return 0;
}

```



```

void calculateTimes (Process p[], int n) {
    int time = 0;
    for (int i = 0; i < n; i++) {
        time += p[i].burst;
        p[i].completion = time;
        p[i].turnaround = p[i].completion;
        p[i].waiting = p[i].turnaround - p[i].burst;
    }

    float calculateUtilization (Process p[], int n) {
        float u = 0;
        for (int i = 0; i < n; i++) {
            u += (float) (p[i].burst / p[i].period);
        }
        return u;
    }

    void check EDF (Process p[], int n) {
        float v = calculateUtilization(p, n);
        printf (" EDF Utilization = %.2f\n", v);
        if (u <= 1.0)
            printf ("Result: SCHEDULABLE (EDF)\n");
        else
            printf ("Result: NOT SCHEDULABLE (EDF)\n");
    }

    void check RMS (Process p[], int n) {
        float u = calculateUtilization(p, n);
        float bound = n * (pow(2, 1.0/n) - 1);
        printf (" RMS Utilization = %.2f\n", u);
        printf (" RMS Bound = %.2f\n", bound);
        printf ("Result: SCHEDULABLE (RMS)\n");
        if (u <= bound)
            printf ("Result: SCHEDULABLE (RMS)\n");
        else
    }
}

```

printf("Result: NOT SCHEDULABLE\n");

void printEDF (process p[j], int n) {

printf("\n == EDF == \n\n");

printf("\n | Period | Deadline | CT | WTT | AT | \n");

for (int i = 0; i < n; i++) {

printf("\n | %d | %d | %d | %d | %d | %d | \n",

p[i].id,

p[i].burst,

p[i].deadline,

p[i].completion,

p[i].waiting,

p[i].turnaround);

}

printf(" - - - - - \n");

}

void printRMS (process p[j], int n) {

printf("\n == RMS == \n\n");

printf("\n | Period | Period | CT | WTT | AT | \n");

for (int i = 0; i < n; i++) {

printf("\n | %d | %d | %d | %d | %d | %d | \n",

p[i].id,

p[i].burst,

p[i].deadline,

p[i].completion,

p[i].waiting,

p[i].turnaround);

}

printf(" - - - - - \n");

}


```
int main() {
```

```
    int n;
```

```
    Process p[MAX], edf[MAX], rms[MAX];
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("\nEnter process details:");
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].id = i + 1;
```

```
        printf("Process ID: ");
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &p[i].burst);
```

```
        printf("Deadline (EDF): ");
```

```
        scanf("%d", &p[i].deadline);
```

```
        printf("Period (RMS): ");
```

```
        scanf("%d", &p[i].period);
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        edf[i] = p[i];
```

```
        rms[i] = p[i];
```

```
    }
```

```
    sort(edf, n);
```

```
    calculateTimes(edf, n);
```

```
    printf("EDF: ");
```

```
    check EDF(p, n);
```

```
    sort(RMS, n);
```

```
    calculateTimes(RMS, n);
```

```
    printf("RMS: ");
```

```
    check RMS(p, n);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
#include <math.h>
#define MAX 10
typedef struct {
    int id;
    int burst;
    int deadline;
    int period;
    int completion;
    int waiting;
    int turnaround;
} Process;

void sortEPF (Process p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].deadline > p[j].deadline) {
                Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

void sortRMS (Process p[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].period > p[j].period) {
                Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```


RMS Utilization = 1.33
RMS Bound = 0.78
Result: NOT SCHEDULABLE (RMS)

Q/P 2

Enter no. of process: 3
Enter process details:

Process 1

BT: 3

Deadline (EDF): 7

Period (RMS): 20

Process 2

BT: 2

Deadline (EDF): 4

Period (RMS): 5

Process 3

BT: 2

Deadline (EDF): 8

Period (RMS): 10

= EDF

ID	BF	Deadline	CT	WT	AT
101	131	7	1	5	1
111	121	4	1	2	0
121	121	8	1	7	1

EDF Utilization = 0.75

Result: SCHEDULABLE

$\Sigma RMSE =$

Job	BF	Period	CT	WT	AT
10	1	3	1	2	1
10	1	3	1	4	1
11	1	2	1	0	1
12	1	2	1	0	1
12	1	2	1	2	1
12	1	2	1	4	1

RMS Utilization = 0.75
Result: Schedulable.

Ex 2
2/13

The following processes
S2=0.5
will print
as P0, P1

Process
while (1)
wait
pri
sig
sig

4

Initial

P0

P1

P2

P0

P1

The

Consider two processes A and B. Two semaphore variables S and T are used to synchronize the processes. S is initialized to 0 and T is initialized to 1. Processes scheduled in the following order: A B A B B A B A A B A

What will be printed on the screen?

Process A:

```
while(1)
{
    wait(S);
    print('P');
    print('P');
    signal(T);
}
```

Process B:

```
while(1)
{
    wait(T);
    print('T');
    print('T');
    signal(S);
}
```

Initial

A

B

A

B

B

A

A

B

A

Initial

A

B

A

B

B

A

A

B

A

Initial	S=0	T=1	Remark
A	0	1	PP
B	1	0	PPIT
A	0	1	PPITPP
B	1	0	PPITPPI
B	1	0	PPITPPIII
A	0	1	PPITPPIIAPP
A	0	1	PPITPPI
B			
A			

Initial	S=0	T=1	Remark
A	0	1	Process A is Blocked (wait)
B	1	0	II
A	0	1	IIPP
B	1	0	IIPII
B	1	0	process B is Blocked (wait)
A	0	1	IIPIIAPP
A	0	1	process B is Blocked (wait)
B	1	0	IIPIIPII
A	0	1	IIPIIPIIAPP

(AT)
2 1
2 1
4 1

The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0=1, S_1=0, S_2=0$ find out how many times Process P0 will print "0". Assume the order of execution as P0, P1, P2, P0, P1.

```

Process P0:           Process P1:           Process P2:
while (true) {         wait(S1);              wait(S2);
wait(S0);              signal(S0);                                signal(S0);
print("0");
signal(S1);
signal(S2);
}
    
```

Initial $S_0=1$ $S_1=0$ $S_2=0$ Remark

P0	0	1	1	0
P1	1	0	1	-
P2	1	0	0	-
P0	0	1	1	0
P1	1	0	1	-

The P0 will print "0" two times.

Dining Philosopher

Philosopher 0 is Thinking
 Philosopher 3 is Thinking
 Philosopher 2 is Thinking
 Philosopher 4 is Thinking
 Philosopher 1 is Thinking
 Philosopher 3 picked up right fork 4
 Philosopher 3 picked up left fork 3
 Philosopher 3 is eating
 Philosopher 0 picked up left fork 0
 Philosopher 0 picked up right fork 1
 Philosopher 0 is eating
 Philosopher 1 picked up right fork 2
 Philosopher 0 put down forks 0 and 1
 Philosopher 3 put down forks 3 and 4
 Philosopher 1 picked up left fork 1
 Philosopher 1 is eating
 Philosopher 2 is Thinking
 Philosopher 0 is Thinking
 Philosopher 3 put down forks 3 and 4
 Philosopher 1 put down forks 1 and 2
 Philosopher 1 picked up left fork 2
 Philosopher 2 picked up right fork 3
 Philosopher 2 picked up left fork 0
 Philosopher 2 is eating
 Philosopher 0 picked up left fork 1
 Philosopher 0 picked up right fork 1
 Philosopher 0 is eating
 Philosopher 3 is Thinking
 Philosopher 1 is Thinking
 Philosopher 2 put down forks 2 and 3
 Philosopher 3 picked up right fork 4

Buffer State

[0, 1, 2]

[0, 1, 2]

[0, 1, 2]

[0, 1, 2, 3]

[0, 1, 2, 3, 4]

[0, 1, 2, 3, 4]

Philosopher 3 picked up left fork 3.
 Philosopher 3 is eating.
 Philosopher 0 put down forks 0 and 1.
 Philosopher 1 picked up right fork 2.
 Philosopher 1 picked up left fork 1.
 Philosopher 1 is eating.
 Philosopher 2 is thinking.
 Philosopher 0 has finished.
 Philosopher 3 put down forks 3 and 4.
 Philosopher 1 put down forks 1 and 2.
 Philosopher 2 picked up left fork 3.
 Philosopher 2 picked up right

✓

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <semaphore.h>

#define N 5

#define LEFT 0

#define RIGHT 1

pthread_t

pthread_t

void *ph

int id

if (id

printf

return

y

int left

int right

for (int

printf

sleep

if (id

ph

ph

Dining Philosopher

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5
#define Active Philosophers 4
#define MAX_CYCLES 3

pthread_mutex_t forks[NUM_PHILOSOPHERS];
pthread_t philosophers[NUM_PHILOSOPHERS];

void *philosopher(void *num) {
    int id = *(int *)num;

    if (id >= ACTIVE_PHILOSOPHERS) {
        printf("Philosopher %d is inactive.\n", id);
        return NULL;
    }

    int left = id;
    int right = (id + 1) % NUM_PHILOSOPHERS;

    for (int i = 0; i < MAX_CYCLES; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up fork %d.\n", id, left);
        }
    }
}
```

Output

consumed: 0 from butter 0.17

Barlored: 1 at butterfly

transposed	at	buffer [27]
------------	----	-------------

Produced: 3 at 1000000

Produced: 4 at butter 4

... d. A from buffer

Tracing

Buffer: (, , ,) , empty

row number index = 0

Step operation

Step	Operation	Item	Buffer Index	Buffer
1	Produce	0	0	0
2	Consume	0	0	-
3	Produce	1	1	0, 1
4	Consume	1	1	0, 1
5	Produce	2	2	0, 1, 2
6	Consume	2	2	-
7	Produce	3	3	0, 1, 2, 3
8	Produce	4	4	0, 1, 2, 3, 4
9	Consume	3	3	0, 1, 2, 3
10	Consume	4	4	-

November

philosophy

philosophy

number

Phylloporinae

Blutdruckher

substance

philosopher

100

Philoxaphes

Date: / /

Phalloidin

philosophy

Philosophy

 $\phi(\Delta) \approx 0$

addition

philosophy

1000

Philosophy


```

for (int i=0; i<NUM_PHILOSOPHERS; i++)
    ids[i] = i;

pthread_create(&philosophers[i], NULL,
    philosopher, &ids[i]);

for (int i=0; i<NUM_PHILOSOPHERS; i++)
    pthread_join(philosophers[i], NULL);

for (int i=0; i<NUM_PHILOSOPHERS; i++)
    pthread_mutex_destroy(&forks[i]);

printf("All philosophers have finished\n");
return 0;
    
```

```
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right\n", id, right);
else
{
```

```
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right\n", id, right);
}
```

```
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left\n", id, left);
}
```

```
printf("Philosopher %d is eating.\n", id);
sleep(2);
```

```
pthread_mutex_unlock(&forks[left]);
```

```
pthread_mutex_unlock(&forks[right]);
```

```
printf("Philosopher %d put down forks\n", id, left, right);
```

```
sleep(1);
}
```

```
printf("Philosopher %d has finished\n", id);
return NULL;
}
```

```
int main() {
```

```
int ids[NUMBER_OF_PHILOSOPHERS];
```

```
for (int i = 0; i < NUMBER_OF_PHILOSOPHERS; i++)
```

```
{ pthread_mutex_init(&forks[i], NULL);
```

```
}
```

```
for (int i = 0; i <
```

```
ids[i];
```

```
pthread
```

```
philosopher
```

```
{
```

```
for (int i = 0;
```

```
pthread
```

```
{
```

```
for (int i =
```

```
pthread
```

```
{
```

```
printf("All
```

```
return 0
```

```
}
```



```

for (int i = 0; i < MAX_ITEMS; i++) {
    next_produced = produced_item(i);
    sem_wait(&empty);
    sem_wait(&mutex);
    buffer[in] = next_produced;
    printf("Produced: %d at buffer[%d]\n",
        next_produced, in);
    in = (in + 1) % Buffer_size;
    sem_post(&mutex);
    sem_post(&full);
    usleep(100000);
}

```

return NULL;

3

void * consumer (void * arg) {

int next_consumed;

for (int i = 0; i < MAX_ITEMS; i++) {

sem_wait(&full);

sem_wait(&mutex);

next_consumed = buffer[out];

int index = out;

out = (out + 1) % Buffer_size;

sem_post(&mutex);

sem_post(&empty);

consume_item(next_consumed, index);

usleep(150000);

3

return NULL;

3

```
int main() {  
    pthread_t prod, cons;  
    sem_init(&empty, 0, BUFFER_SIZE);  
    sem_init(&full, 0, 0);  
    sem_init(&mutex, 0, 1);  
  
    pthread_create(&prod, NULL, producer, NULL);  
    pthread_create(&cons, NULL, consumer, NULL);  
    pthread_join(prod, NULL);  
    pthread_join(cons, NULL);  
    sem_destroy(&empty);  
    sem_destroy(&full);  
    sem_destroy(&mutex);  
    return 0;  
}
```

Consider
system:

P

A

P0

P1

P2

P3

P4

Total

P0

Finish

N

0

W

P1

Finish

N

P2

Consider the following snapshot of a system:-

	Allocation				Max Available				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0	0	7	5	0
P2	1	3	5	4	2	3	5	6	1	0	0	2
P3	0	6	3	2	0	6	5	2	0	0	7	0
P4	0	0	1	4	0	6	5	6	0	6	4	2
Total	2	9	10	12								
	1	5	2	0								
	3	14	12	12								

Σ P_{0j}

P₀

finish[0] = false

Need ≤ available

$$0000 \leq 1520$$

$$Work = Work + Allocation$$

$$= 1520 + 0012$$

$$= 1532$$

finish[0] = true

P₁

finish[1] = false

Need ≤ available

$$0750 \leq 1532$$

$$Work = Work + Allocation \quad finish[1] = false$$

$$= 1532 +$$

P₂ finish[2] = false

Need ≤ available

$$1002 \leq 1532$$

$$Work = Work + Allocation$$

Bounded buffer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 5
#define MAX_ITEMS 5
```

```
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

```
sem_t empty;
sem_t full;
sem_t mutex;
```

```
int produce_item(int i) {
    return i;
}
```

```
void consume_item(int item, int index) {
    printf("Consumed: %d from buffer [%d]\n",
           item, index);
}
```

```
void * producer(void * arg) {
    int next_produced;
```

```
for (int i = 0; i < MAX_ITEMS; i++) {
    int item = produce_item(i);
    sem_wait(&empty);
    sem_wait(&mutex);
    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    sem_post(&mutex);
    printf("Produced: %d\n", item);
    next_produced = i + 1;
}
```


Code

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int main() {
    int m, n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter no. of resources: ");
    scanf("%d", &m);
    int alloc[MAX][MAX], max[MAX][MAX],
    need[MAX][MAX];
    int available[MAX];
    printf("Enter allocation matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("Enter maximum demand matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
    printf("Enter available resources:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &available[i]);
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
}
```

$$= 1532 + 1354$$

$$= 2886$$

$$\text{finish}[2] = \text{true}$$

$$P3 \quad \text{finish}[3] = \text{false}$$

$$\text{Need} \leq \text{available}$$

$$0020 \leq 2886$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 2886 + 0632$$

$$= 214118$$

$$\text{finish}[3] = \text{true}$$

P4

$$\text{finish}[4] = \text{false}$$

$$\text{Need} \leq \text{available}$$

$$0642 \leq 214118$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 214118 + 0014$$

$$= 214132$$

$$\text{finish}[4] = \text{True}$$

P1

$$\text{finish}[1] = \text{false}$$

$$\text{Need} \leq \text{available}$$

$$0750 \leq 214112$$

$$\text{work} = \text{work} + \text{allocation}$$

$$= 214112 + 1000$$

$$= 314112$$

8

∴ The safety safe sequence is

∅ P0, P2, P3, P4, P1

Signature

printf("A\nSystem is not in a safe state
procedure possible\n");
return 0;

printf("C\nSystem is in safe state.\n");
printf("safe sequence is: ");
for (int i = 0; i < n; i++)
printf("%d", safeSeq[i]);
if (i != n-1);
printf("\n");
return 0;

// New request

int process;
int request[MAX];

printf("Enter process number making
request: ");
scanf("%d", &process);
printf("Enter request vector: ");
for (int i = 0; i < m; i++)
scanf("%d", &request[i]);
for (int i = 0; i < m; i++)
if (request[i] > need[process][i])
printf("Error: Request exceeds
maximum need.\n");
return 0;

```

for(int i=0; i<m; i++)
{
    if(request[i] > available[i])
    {
        printf("\n Resources not available.  
Process must wait.\n");
        return 0;
    }
    for(int j=0; j<n; j++)
    {
        available[j] -= request[i][j];
        alloc[process][i] += request[i][j];
        need[process][i] -= request[i][j];
    }
}

```

O/P

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:

0	1	0
3	0	2
3	0	2
2	1	1
0	0	2

Enter Maximum Demand Matrix,

7	4	3
0	2	0
6	0	0
0	1	1
4	3	1

Enter
2 3 0

Enter
3 3

Enter

Enter
0 0

Req
Sys
Suf

Enter Available Resources
2 3 0

Enter process number making request: 4
Enter request vector:
3 3 0

Error: Request Exceeds maximum need

or,

Enter process number making request: 0
Enter request vector:
0 2 0

Request can be granted.
System is in SAFE state.
Safe sequence: P3 → P1 → P2 → P0 → P4.

```
need[i][j] = max[need[i][j-alloc[i][j]];
```

```
int work[Max], safe[Max];
```

```
bool finish[Max];
```

```
for (int i = 0; i < m; i++)
```

```
    work[i] = available[i];
```

```
for (int i = 0; i < m; i++)
```

```
    finish[i] = false;
```

```
int count = 0;
```

```
while (count < n) {
```

```
    bool found = false;
```

```
    for (int i = 0; i < m; i++)
```

```
        if (!finish[i]) {
```

```
            bool possible = true;
```

```
            for (int j = 0; j < m; j++) {
```

```
                if (need[i][j] > work[j]) {
```

```
                    possible = false;
```

```
                    break;
```

```
                }
```

```
            } if (possible) {
```

```
                for (int j = 0; j < m; j++) {
```

```
                    work[j] += alloc[i][j];
```

```
                } safe[seq(count++)] = i;
```

```
                finish[i] = true;
```

```
                found = true;
```

```
            }
```

```
        }
```

```
    }
```

```
    if (!found) {
```



```
for (int i = 0; i < n; i++) {
    int flag = 0;
    for (int j = 0; j < m; j++) {
        if (alloc[i][j] != 0) {
            flag = 1;
            break;
        }
    }
}
```

```
finish[i] = (flag == 0) ? 1 : 0;
}

for (int k = 0; k < n; k++) {
    for (int i = 0; i < m; i++) {
        if (finish[i] == 1) {
            int j;
            for (j = 0; j < m; j++) {
                if (request[i][j] > work[k][j])
                    break;
            }
            if (j == m) {
                for (int n = 0; n < m; n++) {
                    work[k][n] += alloc[i][n];
                }
                finish[i] = 1;
            }
        }
    }
}
```

```
int deadlock = 0;
printf("\nSystem Deadlock Analysis:\n");
```

```

for(int i=0; i<n; i++) {
    if (!finish[i]) {
        if (deadlock == 0) {
            printf("Deadlock detected\n");
            printf("Processes causing\n");
            printf("deadlock: ");
            printf("P%d", i);
            deadlock = 1;
        }
        if (!deadlock) {
            printf("No deadlock detected. All processes\n");
            printf("can finish.\n");
        }
        else {
            printf("\n These processes are stuck and\n");
            printf("waiting for each other.\n");
        }
    }
}

```

X - - - X

O/P
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation Matrix:

0	1	0
2	0	0
3	0	3
2	1	1
0	0	2

Enter Request
0 0 0
2 0 2
0 0 1
1 0 0
0 0 2
Enter avail
5 0 0
System Del
deadlock
Processes
These pro
for exit

#Deadlock detection

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, m, j;
    int alloc[MAX][MAX], request[MAX][MAX],
    int available;
    int work[MAX];
    int finish[MAX];
    printf("Enter no of processes: ");
    scanf("%d", &n);
    printf("Enter no of resource types: ");
    scanf("%d", &m);
    printf("\nEnter Allocation Matrix: ");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("\nEnter Request Matrix: ");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &request[i][j]);
        }
    }
    printf("\nEnter Available Resources: ");
    for (int i = 0; i < m; i++) {
        work[i] = available[i];
    }
}
```


Memory allocation

- First fit fit
- Best fit fit
- Worst fit fit

Given a free memory partition
400K, 200K, 300K and 600K how would each
of first fit, best fit and worst fit
algorithms work to place processes of
212K, 512K, 312K, 526K (in order)

First fit

212K	512K		312K
------	------	--	------

→ 400K → 700K → 200K → 300K → 600K →

P₁ → 212K

P₂ → 512K

P₃ → 312K

P₄ → 526K

Best fit

312K	526K	212K	512K
------	------	------	------

→ 400K → 700K → 200K → 300K → 600K →

Worst fit

312K	512K	212K	512K
------	------	------	------

→ 400K → 700K → 200K → 300K → 600K →

526K cannot be allocated with the memory.

Given five memory partitions of 300KB, 600KB, 350KB, 200KB, 250KB (in order). How would each of first, best, worst fit algorithm work to place process of 115KB, 500KB, 250KB, 200KB, 375KB (in order)?

- P1 = 115KB
- P2 = 500KB
- P3 = 375KB
- P4 = 200KB
- P5 = 375KB

First fit

115KB	500KB	200KB	350KB	250KB
300KB	600KB	350KB	200KB	115KB
300KB	600KB	350KB	200KB	115KB

Best fit

375KB cannot be allocated with memory

Best fit

500KB	200KB	350KB	115KB
300KB	600KB	350KB	200KB
300KB	600KB	350KB	200KB

375KB cannot be allocated with memory

Worst fit

500KB	200KB	115KB
300KB	600KB	350KB
300KB	600KB	350KB

```

350KB
main
#include
int main
printf
scanf
int b
printf
for cin
scanf
2
printf
scanf
int p
printf
for cin
scanf
2
int b
for cin
blo
blo
blo
2
first
best
worst
return
    
```

248 300KB
first bit
of 115KB
KB

125KB

memory

115KB

125KB

memory

125KB

355KB and 375KB cannot be allocated with memory.

Code for contiguous memory allocation:

```
#include <stdio.h>
```

```
int main() {
```

```
int m,n;
```

```
printf("Enter number of memory blocks: ");
```

```
scanf("%d",&m);
```

```
int blockSize[m];
```

```
printf("Enter sizes of memory blocks:\n");
```

```
for (int i=0; i<m; i++) {
```

```
scanf("%d",&blockSize[i]);
```

```
}
```

```
printf("Enter number of processes ");
```

```
scanf("%d",&n);
```

```
int processSize[n];
```

```
printf("Enter sizes of processes:\n");
```

```
for (int i=0; i<n; i++) {
```

```
scanf("%d",&processSize[i]);
```

```
}
```

```
int block1[m],block2[m],block3[m];
```

```
for (int i=0; i<m; i++) {
```

```
block1[i]=blockSize[i];
```

```
block2[i]=blockSize[i];
```

```
block3[i]=blockSize[i];
```

```
}
```

```
firstFit(block1,m,processSize,n);
```

```
bestFit(block2,m,processSize,n);
```

```
worstFit(block3,m,processSize,n);
```

```
return 0;
```

```
}
```


Enter Request Matrix:

0 0 0
2 0 2
0 0 1
1 0 0
0 0 2

Enter available Resources:

0 0 0

System Deadlock Analysis:

Deadlock detected!

Processes causing deadlock: P1 P2 P3 P4
These processes are stuck and waiting
for each other.

~~Stop~~

```
void bestfit (int blockSize[], int m, int process[], int n) {
    int allocation[n];
    for (int i = 0; i < m; i++)
        allocation[i] = -1;
    for (int l = 0; l < n; l++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= process[l]) {
                if (bestIdx == -1 || blockSize[j] <
                    blockSize[bestIdx])
                    bestIdx = j;
            }
        }
    }
}
```

```
if (bestIdx != -1) {
    allocation[i] = bestIdx;
    blockSize[bestIdx] -= process[l];
}
```

```
printf("\n BEST FIT Allocation:\n");
printf("Process No.\tProcess Size\tBlock no.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i+1, process[i],
        if (allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("Not Allocated\n");
    }
}
```

```
void worstFit (int blockSize[], int m,
int processSize[], int n) {
    int allocation[n];
```

```
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
```

```
    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (blockSize[j] >= processSize[i] &&
```

```
                if (worstIdx == -1 || blockSize[j] >
                    blockSize[worstIdx])
                    worstIdx = j;
```

```
        }
```

```
    }
```

```
    if (worstIdx != -1) {
```

```
        allocation[worstIdx] = processSize[i];
```

```
        blockSize[worstIdx] -= processSize[i];
```

```
    }
```

```
    }
```

```
    printf("\n Worst FIT Allocation:\n");
```

```
    printf("Process No. \t Process Size \t Block No. \t");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d \t %d \t %d \t", i+1, processSize[i],
```

```
            if (allocation[i] != -1)
```

```
                printf("%d", allocation[i]+1);
```

```
            else
```

```
                printf("Not Allocated\n");
```

```
    }
```

```
    }
```



```
void firstFit(int blockSize[], int m, int processSize,
int n) {
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }
}
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
int findOptimal (int pages[], int n, int frameCount, int index) {
```

```
    int farthest = index, pos = -1;
```

```
    for (int i = 0; i < frameCount; i++) {
```

```
        int j;
```

```
        for (j = index; j <= n; j++) {
```

```
            if (frame[i] == pages[j]) {
```

```
                if (j < farthest) farthest = j;
```

```
                pos = j;
```

```
            }
```

```
        }
```

```
        break;
```

```
    }
```

```
    }
```

```
    if (j == n) return j;
```

```
    }
```

```
    return (pos == -1) ? 0 : pos;
```

```
    }
```

```
int findLRU (int time[], int frameCount)
```

```
{
```

```
    int main = time[0], pos = 0;
```

```
    for (int i = 1; i < frameCount; i++) {
```

```
        if (time[i] < main) {
```

```
            min = time[i];
```

```
            pos = i;
```

```
        }
```

```
    }
```

```
    return pos;
```

```
}
```

void FIFO (int pages[], int n, int frameCount)

{
 int frame[frameCount]; count = 0; faults = 0;

for (int i = 0; i < frameCount; i++) frame[i] = -1;

for (int i = 0; i < n; i++)

 int found = 0;

 for (int j = 0; j < frameCount; j++)

 if (frame[j] == pages[i]) { found = 1;

 break;

 }

 if (!found)

 frame[count] = pages[i];

 count = (count + 1) % frameCount;

 faults++;

 }

 printf("FIFO page faults: %d\n", faults);

}

void LRU (int pages[], int n, int frameCount)

{
 int frame[frameCount]; time[frameCount];

 faults = 0; counter = 0;

 for (int i = 0; i < frameCount; i++) frame[i] = -1;

 for (int i = 0; i < n; i++)

 int found = 0;

 for (int j = 0; j < frameCount; j++)

 if (frame[j] == pages[i])

 found = 1;

 time[j] = counter;

 found = 1;

 break;

 }

 if (!found)

 frame[count] = pages[i];

 count = (count + 1) % frameCount;

 faults++;

 }

 }

 printf("LRU page faults: %d\n", faults);

}

Enter number of memory blocks: 5
Enter sizes of memory blocks: 400 700 200 300 600

Enter number of processes: 4
Enter sizes of processes:

212

512

312

526

FIRST FIT Allocation

Process No.	Process Size	Block no.
1	212	1
2	512	2
3	312	5
4	526	Not Allocated

BEST FIT Allocation

Process No.	Process Size	Block no.
1	212	4
2	512	5
3	312	1
4	526	2

Worst FIT Allocation

Process No.	Process Size	Block no.
1	212	5
2	512	2
3	312	Not Allocated
4	526	Not Allocated

[Signature]
12/11

```

}
j, break;
}
frames[n] = 1;
}
int
-0;
frames[n] = 1;
}
+2
}
j, break;
}
ages[n]

```

```

faults++;
}
}
print ("Optimal pages faults: %d\n", faults);
}
int main() {
    int n, frameCount;
    printf ("Enter number of pages: ");
    scanf ("%d", &n);
    int pages[n];
    printf ("Enter the reference string: ");
    for (int i = 0; i < n; i++) scanf ("%d", &pages[i]);
    printf ("Enter number of frames: ");
    scanf ("%d", &frameCount);
    FIFO (pages, n, frameCount);
    LRU (pages, n, frameCount);
    optimal (pages, n, frameCount);
    return 0;
}

```

y
O/p
Enter number of pages: 20
Enter the reference string: 7012030423032120301
Enter number of frames: 3
FIFO Page Faults: 15
LRU Page Faults: 12
Optimal Page Faults: 9

7 0 1
7 7 7
0


```

if (!found) {
    int pos = -1;
    for (int j = 0; j < frameCount; j++) {
        if (frame[j] == -1) {
            pos = j; break;
        }
    }
    if (pos == -1) pos = findLRU(time, frameCount,
        frame[pos] = pages[i],
        counter++;
    time[pos] = counter;
    faults++;
}

```

```

printf("LRU page faults: %d\n", faults);

void Optimal(int pages[], int n, int
    frameCount) {
    int frame[frameCount], faults = 0;
    for (int i = 0; i < frameCount; i++) frame[i] = -1;
    for (int i = 0; i < n; i++) {
        if (frame[i] == -1) {
            found = 1; break;
        }
    }
    if (!found) {
        int found = 0;
        for (int j = 0; j < frameCount; j++) {
            if (frame[j] == pages[i]) {
                found = 1; break;
            }
        }
        if (!found) {
            int pos = -1;
            for (int j = 0; j < frameCount; j++) {
                if (frame[j] == -1) {
                    pos = j; break;
                }
            }
            if (pos == -1) pos = findOptimal(pages, n,
                frame, frameCount, i + 1);
            frame[pos] = pages[i];
        }
    }
}

```

```

if (!found) {
    int pos = -1;
    for (int j = 0; j < frameCount; j++) {
        if (frame[j] == -1) {
            pos = j; break;
        }
    }
    if (pos == -1) pos = findOptimal(pages, n,
        frame, frameCount, i + 1);
    frame[pos] = pages[i];
}

```

```

if (pos == -1) pos = findOptimal(pages, n,
    frame, frameCount, i + 1);
frame[pos] = pages[i];
}

```

faults++
 }
 }
 print ("LRU page faults: %d\n", faults);
 int main()
 {
 int n;
 printf("Enter number of pages: ");
 scanf("%d", &n);
 int frameCount;
 printf("Enter number of frames: ");
 scanf("%d", &frameCount);
 int pages[n];
 printf("Enter pages: ");
 for (int i = 0; i < n; i++) {
 scanf("%d", &pages[i]);
 }
 int faults = 0;
 int optimalFaults = 0;
 printf("LRU page faults: ");
 Optimal(pages, n, frameCount);
 printf("Optimal page faults: ");
 Optimal(pages, n, frameCount);
 return 0;
 }

Page 0 - 1701
 Page 1 - 1701
 Page 2 - 1701
 Page 3 - 1701
 Page 4 - 1701

Total page 1701

1701